

Техническое задание на MVP-платформу ShopCAP

1. Общая информация

Название проекта: ShopCAP

Формат: pet-проект / MVP Web3-маркетплейса с кешбеком на Solidity

Цель MVP: показать работающий прототип платформы, где пользователь может подключить кошелек, выбрать товар или бренд-партнера, выполнить симулированную покупку и получить кешбек в токенах SCAP.

ShopCAP позиционируется как consumer cashback engine: пользовательская покупка генерирует ценность для платформы, а часть этой ценности возвращается пользователю в токенизированном виде.

2. Бизнес-идея проекта

Платформа интегрирует бренды и партнеров, а пользователи получают кешбек за покупки.

Для MVP основной акцент делается не на полноценный e-commerce checkout, а на демонстрацию трех вещей:

1. маркетплейс существует и им можно пользоваться;
2. покупка вызывает on-chain-логику;
3. после покупки автоматически распределяется кешбек по фиксированным правилам.

Дополнительная идея проекта:

- бренды могут платить за размещение, рекламу и партнерские интеграции;
- денежный поток платформы направляется в токеномику;
- в будущем токен может использоваться для governance, бустов кешбека и голосования за интеграции.

Для MVP эти расширения не реализуются в полном объеме и фиксируются как post-MVP roadmap.

3. Цели MVP

3.1 Главная цель

Собрать минимально жизнеспособный Web3-продукт, который можно:

- открыть по ссылке;
- подключить через MetaMask;
- "потыкать" как маркетплейс;
- показать работающий смарт-контрактный flow начисления кешбека.

3.2 Что должно увидеть проверяющее лицо

- понятный лендинг/маркетплейс;
- подключение кошелька;
- список партнеров или товаров;
- возможность совершить демо-покупку;
- начисление SCAP после покупки;
- прозрачную логику распределения кешбека через контракты;
- GitHub-репозиторий с понятной архитектурой.

4. Границы MVP

4.1 Что входит в MVP

- ERC-20 токен ShopCAPToken (SCAP) ;
- реестр партнеров;
- центральный контракт платформы;
- контракт расчета и распределения кешбека;
- фронтенд с подключением кошелька;
- отображение товаров/брендов;
- симуляция покупки;
- начисление кешбека на адрес пользователя;

- отображение баланса и истории покупок.

4.2 Что сознательно упрощается

- нет полноценной интеграции с реальным магазином;
- нет реального оракула подтверждения покупки;
- нет голосования за добавление брендов;
- нет динамических множителей кешбека;
- нет полноценной back-office системы;
- нет off-chain order management;
- нет real-time price oracle.

4.3 Что не входит в MVP

- продвинутая ролевая модель;
- multisig;
- upgradeable proxy;
- staking;
- LP/DeFi интеграции;
- полноценная аналитика брендов;
- сложная реферальная многоуровневая система.

5. Роли пользователей

5.1 Администратор платформы

Администратор:

- деплоит контракты;
- настраивает адреса контрактов;
- добавляет партнеров;
- добавляет товары на витрину;
- может обновлять параметры кешбека;

- управляет резервным адресом платформы;
- контролирует демонстрационный сценарий MVP.

5.2 Пользователь

Пользователь:

- подключает MetaMask;
- просматривает товары/партнеров;
- совершает демо-покупку;
- получает кешбек в SCAP ;
- видит свой баланс и историю покупок.

5.3 Партнер

Партнер:

- регистрируется администратором;
- имеет описание, referral link и адрес кошелька;
- участвует в витрине маркетплейса;
- может получать value от покупательского трафика и реферальной модели.

6. Основная бизнес-логика

6.1 Базовый кешбек

При покупке формируется кешбек, равный 1% от суммы покупки.

Формула:

```
cashbackPool = purchaseAmount * 1 / 100
```

6.2 Распределение кешбека

Полученный кешбек распределяется по фиксированной схеме:

- 70% от кешбек-пула начисляется пользователю;

- 20% от кешбек-пула направляется в резерв маркетплейса;
- 10% от кешбек-пула сжигается.

Формулы:

- $\text{userReward} = \text{cashbackPool} * 70 / 100$
- $\text{reserveReward} = \text{cashbackPool} * 20 / 100$
- $\text{burnReward} = \text{cashbackPool} * 10 / 100$

6.3 Специальный кейс для администратора

Если покупку совершает сам администратор платформы, то его награда должна отличаться от обычного пользователя:

- пользовательская часть 70% остается у него;
- резервная часть 20% также фактически принадлежит ему как маркетплейсу;
- 10% по-прежнему сжигается.

Итог для админа:

- 90% кешбек-пула получает адрес администратора;
- 10% отправляется на burn address.

Формула:

- $\text{adminReward} = \text{cashbackPool} * 90 / 100$
- $\text{burnReward} = \text{cashbackPool} * 10 / 100$

Это требование должно быть явно реализовано в бизнес-логике `CashbackManager` или в центральном контракте платформы.

6.4 Симуляция покупки

В MVP покупка не обязана подтверждаться реальным маркетплейсом или реальным off-chain заказом.

Допускается симулированный flow:

1. пользователь нажимает `Buy` ;
2. фронтенд вызывает on-chain функцию покупки;
3. контракт считает сумму покупки;
4. контракт начисляет кешбек;
5. пользователь видит обновленный баланс.

Ключевая задача MVP: доказать работу модели маркетплейса и кешбек-логики, а не реализовать полноценный e-commerce backend.

7. Архитектура смарт-контрактов

7.1 ShopCAPToken.sol

Назначение:

- реализация ERC-20 токена `SCAP` ;
- `mint` для выпуска наград;
- хранение базовой токеномики.

Требования:

- стандарт OpenZeppelin ERC-20;
- `18 decimals`;
- `owner/minter access control` для `mint`;
- возможность назначать `minter`-адреса через `setMinter` .

Ожидаемое применение в MVP:

- `mint` пользователю на сумму кешбека;
- `mint` в резервный адрес;
- `mint` на `burn address` для учета сжигания.

7.2 PartnerRegistry.sol

Назначение:

- хранение списка партнеров;

- выдача данных по партнеру;
- управление активностью партнеров.

Структура партнера:

- `id`;
- `isActive`;
- `name`;
- `description`;
- `referralLink`;
- `partnerWallet`.

Обязательные функции:

- `addPartner(...)`;
- `updatePartner(...)`;
- `togglePartnerStatus(...)`;
- `getPartnerDetails(id)`;
- `getPartnerWallet(id)`.

Требование для MVP:

- добавление и редактирование партнера выполняет только администратор.

7.3 CashbackManager.sol

Назначение:

- расчет кешбека;
- распределение награды;
- хранение процентных настроек;
- поддержка резервного адреса;
- опциональная поддержка реферального партнера.

Основные параметры:

- `cashbackBasePercent = 1;`
- `userCashbackShare = 70;`
- `reserveShare = 20;`
- `burnShare = 10;`
- `BURN_ADDRESS = 0x00000000000000000000000000000000dEaD.`

Обязательная логика:

- расчет кешбек-пула от суммы покупки;
- mint награды пользователю;
- mint в резерв;
- mint на burn address;
- отдельная ветка для случая, когда покупатель является админом платформы.

7.4 ShopCAPPlatform.sol

Назначение:

- центральная точка взаимодействия;
- управление товарами;
- маршрутизация вызовов к `PartnerRegistry` и `CashbackManager`;
- публичная точка входа для покупки.

Структура товара:

- `id;`
- `name;`
- `price;`
- `stock;`
- `partnerId;`
- `isActive.`

Обязательные функции:

- `listItem(...)` ;
- `buyItem(itemId)` ;
- `addPartner(...)` ;
- `getAllItems()` ;
- `withdraw()` .

Требования:

- добавление товара только администратором;
- добавление партнера только администратором;
- покупка доступна пользователю через фронтенд;
- при покупке платформа должна вызывать кешбек-логику.

8. Функциональные требования

8.1 Регистрация и подключение

Система должна:

- позволять подключить MetaMask;
- определять активный адрес пользователя;
- проверять, что пользователь находится в поддерживаемой сети;
- отображать ошибку при неверной сети.

8.2 Работа с партнерами

Администратор должен иметь возможность:

- добавить нового партнера;
- указать название, описание, ссылку и адрес кошелька;
- активировать/деактивировать партнера.

Пользователь должен иметь возможность:

- просматривать список активных партнеров.

8.3 Работа с товарами

Администратор должен иметь возможность:

- создать товар на платформе;
- указать цену;
- указать остаток;
- связать товар с партнером.

Пользователь должен иметь возможность:

- просматривать доступные товары;
- видеть цену;
- инициировать покупку.

8.4 Покупка

Система должна:

- принять вызов покупки;
- проверить, что товар активен;
- проверить наличие на складе;
- уменьшить stock;
- вызвать логику кешбека;
- сохранить событие покупки.

8.5 Начисление награды

После успешной покупки:

- пользователю начисляются токены SCAP ;
- резерву начисляется доля платформы;
- часть отправляется на burn address;
- на фронтенде обновляется баланс пользователя;
- покупка отображается в истории.

8.6 История действий

Во фронтенде необходимо показывать:

- список купленных товаров;
- дату покупки;
- tx hash;
- ссылку на Etherscan для тестовой сети.

9. Нефункциональные требования

9.1 Требования к безопасности

- использовать OpenZeppelin;
- защищать чувствительные методы access control-модификаторами;
- использовать Ownable или AccessControl;
- защищать покупку от reentrancy;
- не давать произвольным адресам вызывать mint-награждение.

9.2 Требования к UX

- интерфейс должен быть достаточно простым для демонстрации;
- пользователь должен понимать, куда нажимать без инструкции;
- лендинг должен объяснять механику в 1-2 экранах;
- баланс токенов и история должны обновляться прозрачно.

9.3 Требования к демонстрации

- проект должен запускаться локально;
- контракты должны быть задеплойены на тестовую сеть;
- фронтенд должен работать с актуальными ABI и адресами;
- должна быть возможность продемонстрировать end-to-end сценарий за несколько минут.

10. Требования к фронтенду

10.1 Основные страницы

MVP должен содержать:

- Home / Marketplace;
- User Dashboard / My Purchases;
- Admin Dashboard;
- Docs;
- About.

10.2 Home / Marketplace

Должен показывать:

- витрину товаров;
- кнопку Buy;
- краткое описание механики кешбека;
- кнопку подключения кошелька или доступ к wallet flow.

10.3 User Dashboard

Должен показывать:

- адрес кошелька;
- баланс SCAP;
- историю демо-покупок;
- ссылки на транзакции.

10.4 Admin Dashboard

Должен позволять:

- добавлять партнера;
- просматривать реестр партнеров;

- добавлять товар на платформу;
- при необходимости запускать тестовые административные действия.

10.5 Интеграция с контрактами

Фронтенд должен:

- использовать `ethers.js`;
- загружать ABI и адреса контрактов из конфигурации;
- корректно работать с Sepolia;
- обрабатывать ошибки кошелька и revert-сообщения.

11. Технический стек

11.1 Blockchain layer

- Solidity 0.8.x
- Hardhat
- Ethers.js v6
- OpenZeppelin Contracts
- TypeScript для deploy/test scripts

11.2 Frontend layer

- React
- React Router
- ethers.js
- MetaMask
- CSS / кастомный UI

12. Сценарий демонстрации MVP

12.1 Подготовка

- ## 1. Администратор деплоит контракты.

2. Во фронтенд подставляются ABI и адреса.
3. Администратор назначает нужные minter-права контрактам, если это требуется реализацией.
4. Администратор добавляет хотя бы одного партнера.
5. Администратор добавляет несколько товаров.

12.2 Демо-сценарий пользователя

1. Пользователь открывает сайт.
2. Подключает MetaMask в Sepolia.
3. Переходит на страницу маркетплейса.
4. Выбирает товар.
5. Нажимает `Buy`.
6. Подтверждает транзакцию.
7. После майнинга транзакции видит начисленный `SCAP`.
8. Переходит в `My Purchases` и видит историю.

13. Критерии приемки

MVP считается выполненным, если:

- разворачиваются все контракты;
- фронтенд подключается к тестовой сети;
- отображается список товаров;
- пользователь может выполнить демо-покупку;
- после покупки начисляется кешбек;
- выполняется распределение `70 / 20 / 10`;
- для администратора корректно работает `special-case` `90 / 10`;
- токены отображаются в балансе;
- история покупки сохраняется и показывается пользователю.

14. Текущее состояние репозитория

По текущему состоянию проекта в репозитории уже присутствуют:

- `contracts/ShopCAPToken.sol`
- `contracts/PartnerRegistry.sol`
- `contracts/CashbackManager.sol`
- `contracts/ShopCAPPlatform.sol`
- deploy-скрипты на Hardhat
- React-фронтенд с роутами `Marketplace`, `My Purchases`, `System Admin`, `Docs`, `About`
- Web3-подключение через MetaMask
- загрузка списка товаров с контракта
- локальная история покупок через `localStorage`

15. Важные замечания по текущей реализации

Ниже перечислены моменты, которые стоит зафиксировать именно как требования на доводку MVP.

15.1 Ограничение доступа

В текущем коде часть методов, которые по смыслу должны быть административными, требует явной фиксации access control:

- добавление партнера;
- обновление партнера;
- переключение статуса партнера;
- добавление товара;
- обновление настроек кешбека;
- вызов начисления кешбека.

В ТЗ эти методы должны считаться `admin-only`.

15.2 Несовпадение purchase flow

Сейчас в проекте есть два разных сценария:

- on-chain `buyItem(...)` в `ShopCAPPlatform`;
- фронтендовая симуляция, которая местами вызывает `issueCashbackAndDistribute(...)` напрямую.

Для итогового MVP нужно оставить один главный сценарий:

- пользователь вызывает `buyItem(...)`;
- внутри него вызывается кешбек-логика.

Прямой вызов `issueCashbackAndDistribute(...)` с фронтенда должен использоваться только если это осознанный демо-режим для администратора.

15.3 Лишняя выдача токенов за сам факт покупки

В текущем `ShopCAPPlatform` присутствует отдельный `mint` токенов пропорционально `msg.value`.

Для концепции кешбек-маркетплейса это нужно явно определить:

- либо это специальная `gamified`-награда и она остается;
- либо для MVP ее надо убрать, чтобы пользователь получал только кешбек от покупки.

Для защиты концепции проекта рекомендуется второй вариант: оставить только кешбек-механику.

15.4 Special-case для администратора

В текущей реализации правило `90% администратору и 10% burn` не выделено как отдельная ветка.

Это должно быть явно добавлено в финальную логику.

15.5 Единицы измерения суммы покупки

В контрактной логике необходимо однозначно зафиксировать:

- `purchaseAmount` передается в `wei`, если покупка оплачивается через `msg.value`;
- либо `purchaseAmount` передается в условных off-chain units.

Для MVP проще и надежнее использовать `wei`.

16. Требования к деплою

Необходимо обеспечить:

- деплой в `Sepolia`;
- хранение RPC и private key в `.env`;
- генерацию фронтенд-конфига с адресами контрактов;
- копирование ABI в фронтенд;
- инструкции по верификации в Etherscan.

17. Post-MVP roadmap

После защиты MVP проект может быть расширен следующими модулями:

- cashback multipliers;
- голосование за интеграцию брендов;
- oracle-подтверждение покупки;
- dynamic cashback per partner;
- role-based access control;
- multisig для владельца;
- upgradeable contracts;
- аналитика брендов;
- staking и utility для SCAP.

18. Итог

ShopCAP MVP должен демонстрировать работающую Web3-модель маркетплейса с токенизированным кешбеком.

Основной акцент не на полном e-commerce цикле, а на том, чтобы через смарт-контракты показать:

- регистрацию партнеров;
- наличие витрины;
- симуляцию покупки;
- прозрачное начисление кешбека;
- понятную токеномику;
- работающий пользовательский интерфейс.

Именно эта версия проекта является оптимальной для рет-проекта и короткого MVP-срока: она достаточно простая для реализации, но уже выглядит как полноценный продукт, который можно показать на GitHub, в демо и на интервью.